

The CodeSheriff Algorithm and the Equation Used by Each Agent

CodeSheriff reviews a pull request one changed function at a time; each changed function is a *unit*. Four independent agents examine the same unit without seeing one another’s answers, and each replies in one of three ways: *found something* (with a score between 0 and 1), *looked and found nothing*, or *could not run*. Those replies become one probability that the change is vulnerable. Every reply is filed under a key built from the file, the function and the weakness class — never the vulnerable line, which each agent would word differently — so that two agents describing one bug are recognised as talking about the same bug:

$$k(u, c) = \text{SHA256}(\text{file} \parallel \text{function} \parallel \text{CWE class } c)_{[0:16]}. \quad (1)$$

Algorithm 1 CODESHERIFF — reviewing one pull request

Require: the changed files; the fitted number table (prior π , alert threshold θ^* , ratios Λ)

Ensure: for every suspected weakness, a probability and a decision whether to alert

```

1: Split the change into units: one unit per changed function
2: for all unit  $u$  do
3:   Send  $u$  to every agent separately. No agent is told what any other agent found
4:   Collect all replies for this unit into  $\mathcal{E}_u$ . An agent that crashes or times out replies could not run
5:   for all distinct weakness  $\kappa$  that some agent reported in  $u$  do
6:     Start from the prior odds: how likely a random changed function is to be vulnerable
7:     for all witness  $W$  of the four do
8:       Read what  $W$  said about  $\kappa$  — found something (strong / medium / weak), found nothing, or could not run — and multiply the
       odds by the matching table entry  $\lambda_W$ . Could not run always gives  $\lambda_W = 1$ 
9:     end for
10:    Convert the odds back to a probability  $p$ , and alert if  $p \geq \theta^*$ 
11:  end for
12: end for
```

Agent 1 — structural. It traces untrusted values through the code without running it. Each node of its graph is a place a value is defined or used, each edge is a value flowing from one place to another, and an edge that sanitises the value is marked with what it makes safe. Every function parameter counts as untrusted, since the signature is where outside data arrives. Let G_c be the graph with every edge that sanitises class c removed. The tainted set T_c is the smallest set containing all parameters and closed under edges of G_c ; a dangerous call is reported only if a real path from a parameter to it exists in G_c . The score rises with how dangerous the call is and falls on a long path or a test file:

$$\varrho_{\text{str}} = \text{clamp}_{[0,1]}(0.45 + \delta(d) - 0.25\varsigma - 0.0125 \min(\max(0, |\pi| - 2), 4) + 0.15\nu - 0.20\tau), \quad (2)$$

with $\delta = 0.20$ for a critical call and 0.10 for a high one, $|\pi|$ the path length, and ς, ν, τ flags for a partial sanitiser, a network-facing unit and a test file.

Agent 2 — semantic. It asks a language model what the function is meant to do, where it trusts its input, and which safety rule the change breaks. The same question is asked $n = 3$ times with different random seeds, and the score is simply how often the model repeats the same finding — agreement with itself is the confidence:

$$\varrho_{\text{sem}} = \frac{\text{number of samples reporting this weakness}}{n}. \quad (3)$$

A model can also invent things, so a finding is kept only when its weakness class is one we handle, its file is exactly this unit’s file, the vulnerable line it quotes really appears in the code, and its line numbers lie inside the unit:

$$\Gamma(\phi, u) = [c(\phi) \in \text{CWE}_{\text{scope}}] \wedge [\text{file}(\phi) = f] \wedge [\text{sink}(\phi) \sqsubseteq s^+] \wedge [\text{lines}(\phi) \subseteq [1, L_u]]. \quad (4)$$

Agent 3 — context. It reads code this repository has already merged and asks whether the change drops a protection the repository normally applies. A protection counts as the repository’s habit only if the same function carried it in an earlier merge, or at least two different functions use it — one neighbour doing it once is a coincidence, not a rule. Anything so established but missing from the new code is reported, scored by how firmly the history supports it:

$$\varrho_{\text{ctx}} = b \cdot \min\left(1, \frac{\bar{s}}{0.85}\right), \quad b = 0.85 \text{ if the same function carried it or } \geq 3 \text{ others do, else } 0.65, \quad (5)$$

where $\bar{s}$ is the mean similarity of the supporting examples.

Agent 4 — runtime. It actually runs the function once inside a WebAssembly sandbox that has no network and no environment, under fixed limits on instructions, time and memory. Every parameter is replaced by a value whose text is a unique random marker, so ordinary string operations carry the marker along for free: a value is untrusted exactly when some marker still appears inside it, $\text{tainted}(v) \iff \exists i : t_i \sqsubseteq v$. An execution either happened or it did not, so there is a single score, $\varrho_{\text{rt}} = 0.85$. A marked value reaching a dangerous operation is reported. If it got there only after passing a check the sandbox could not evaluate, the agent replies *could not run*, since that check may be exactly what makes the code safe. If nothing was reached, it replies *looked and found nothing*.

Combining the four answers. A score of 0.8 or more is *strong*, 0.5–0.8 is *medium*, below that *weak*; these plus *found nothing* are the four table cells. Each witness contributes exactly one multiplier onto the prior odds:

$$\frac{p}{1-p} = \frac{\pi}{1-\pi} \prod_{W \in \mathcal{W}} \lambda_W, \quad \lambda_W = \begin{cases} \text{clamp}_{[0.05, 20]}(\Lambda_W[\text{cell}_W]) & \text{if } W \text{ answered} \\ 1 & \text{if } W \text{ could not run.} \end{cases} \quad (6)$$

Because all four always take part, an agent that looked and found nothing *lowers* the probability, while one that could not run leaves it unchanged. Each multiplier counts, on labelled examples, how much more often a vulnerable case produces that cell than a safe one, smoothed so that a rare cell cannot dominate:

$$\Lambda_W(z) = \frac{P(\text{cell} = z \mid \text{vulnerable})}{P(\text{cell} = z \mid \text{safe})} \approx \frac{(n_1(z) + \alpha)/(N_1 + 4\alpha)}{(n_0(z) + \alpha)/(N_0 + 4\alpha)}, \quad \Lambda_W(z) = 1 \text{ if the cell was never seen.} \quad (7)$$

Ratios come from one data split and θ^* from a second; a third is kept sealed for the final evaluation.